

合肥工业大学

智能无人飞行器设计 设计报告

学	院	计算机与信息学院
专	业	物联网工程 2023 级 1 班
学 生 姓 名 及 学 号		莫仁鹰 2023212167
指 导 教 师		
课 题 名 称		智能无人飞行器的设计与应用

2025 年 6 月 17 日

一、课题概述（四号、宋体）

本课题以"智能无人飞行器的设计与应用"为研究方向，围绕固定翼无人机的组装、调试、智能目标识别与远程控制等核心技术展开研究。课题背景源于无人机技术在军事侦察、农业植保、环境监测等领域的广泛应用需求。

本项目的目标是完成一套完整的无人机系统搭建与智能化改造方案，包括：飞控固件的烧录与参数配置、嵌入式开发板（RK3566 泰山派）的系统部署、基于 YOLOv5 的目标检测模型训练与部署、以及基于 WireGuard VPN 的跨网络远程通信系统设计。通过本课题的实施，掌握无人机系统集成、嵌入式开发、深度学习模型部署及网络通信等关键技术。

二、课题任务

本课题的核心任务如下：

2.1 硬件系统搭建

完成固定翼无人机的机体组装，包括机身框架、电机、螺旋桨、电调、电池等部件的安装与连接。

2.2 飞控系统配置

使用 Mission Planner 软件对 Pixhawk 飞控进行固件烧录、参数配置和航点规划，实现自主飞行功能。

2.3 嵌入式系统部署

对 RK3566 泰山派开发板进行系统镜像烧录，部署 Ubuntu 操作系统和 ROS

机器人操作系统。

2.4 目标识别系统开发

完成数据集标注、YOLOv5 模型训练、模型格式转换（PT→ONNX→RKNN）及模型部署。

2.5 远程通信系统设计

基于阿里云服务器和 WireGuard VPN 技术，实现虚拟机、云服务器、开发板三端跨网络互通。

技术指标：目标检测准确率 $\geq 90\%$ ；VPN 网络延迟 $\leq 50\text{ms}$ ；系统支持远程实时控制。

三、技术方案及关键问题

3.1 技术方案

本课题采用"飞控+嵌入式开发板+云服务器"的三层架构设计：

- 底层：Pixhawk 飞控负责飞行姿态控制和航线执行
- 中层：RK3566 开发板运行 ROS 系统，负责目标识别、数据处理和指令转发
- 上层：阿里云服务器作为 VPN 中继节点，实现跨网络远程控制

开发平台：Ubuntu 22.04 + Python 3.8 + YOLOv5 + RKNN-Toolkit2 + WireGuard

3.2 关键问题及解决方案

(1) USB 数据线识别问题：实验初期使用充电线连接 Pixhawk 飞控，设备管理器无法识别 COM 端口。经排查确认是纯充电线（仅有电源线无数据线），更换为 Micro USB 数据线后解决。

(2) RNDIS 网络适配器问题：RK3566 开发板通过 USB 连接电脑时，Windows 系统的 RNDIS 驱动未正确安装，导致无法通过 USB 进行网络通信。通过手动安装 RNDIS 驱动解决。

(3) 零件丢失问题：组装完成后放置在实验室的零件被其他人员取走，导致验收前两天才重新获得完整部件。

(4) 模型部署格式兼容：YOLOv5 模型需经过 PT→ONNX→RKNN 的格式转换链路，使用 rknn-toolkit2 工具完成。

四、设计实现及测试

4.1 飞机机体组装

将无人机机身框架、电机、螺旋桨、电调、接收机、电池等部件按照设计图纸进行安装和连接。组装过程中需注意电机旋转方向、螺旋桨匹配、重心平衡等关键要素。

组装过程中遇到的问题：最初组装完成后放置在实验室，后被其他人员取走部分零件。在验收前两天获得了完整的替换部件，在小组协作下完成了重新组装。此外，连接泰山派开发板时发现 Type-C 数据线无法被识别，经排查后确认是

数据线质量问题，更换后恢复正常。

【占位符 A：飞机机体组装照片（小组合作拼装照片、摄像头安装、开发板上电等，约 3-5 张）——已有用户实拍照片待插入】

4.2 飞控固件烧录（实验 3.2）

使用 Mission Planner 软件通过 USB 数据线连接 Pixhawk 飞控，完成固件升级、参数加载和航点规划。

操作步骤：

- (1) 使用 Micro USB 数据线连接 Pixhawk 飞控至电脑
- (2) 打开 Mission Planner，选择对应 COM 端口，波特率设置为 115200
- (3) 点击"连接"建立通信
- (4) 进入"初始设置"→"安装固件"，选择对应机型进行固件烧录
- (5) 在"全参数列表"中加载 pixhawk 飞控参数.param 参数文件
- (6) 在"飞行计划"中加载.waypoints 航点文件

【占位符 B：Mission Planner 连接、固件烧录、参数加载、航点规划界面截图（约 3-4 张）——需在本地电脑操作截图】

4.3 RK3566 系统镜像烧录（实验 3.3）

使用 RKDevTool 烧录工具将 Ubuntu+ROS 系统镜像烧录至 RK3566 泰山派开发板。

操作步骤：

- (1) 安装 USB 驱动：运行 DriverAssitant_v5.1.1 中的 DriverInstall.exe
- (2) 打开 RKDevTool_v2.92 烧录工具
- (3) 加载系统镜像文件（从百度网盘下载）
- (4) 使用 Type-C 数据线连接开发板至电脑
- (5) 按住 REC 键并短按 RST 键使开发板进入 Loader 模式
- (6) 识别到 LOADER 设备后点击"执行"开始烧录
- (7) 等待烧录完成，开发板自动重启

4.4 数据标注与数据集构建（实验 6.1）

使用 Make Sense 在线标注工具（makesense.ai）对目标图片进行标注，生成 YOLO 格式的数据集。

操作步骤：

- (1) 访问 makesense.ai 网站，上传训练图片
- (2) 创建目标标签（如：tank 坦克）
- (3) 选择"Object Detection"目标检测模式
- (4) 对每张图片中的目标绘制边界框并分配标签
- (5) 标注完成后导出为 YOLO 格式（.txt 标注文件）
- (6) 按照 YOLOv5 要求的目录结构组织数据集

数据集统计：训练集 613 张图片，验证集 218 张图片，共 831 张标注图片。

4.5 模型训练（实验 6.2）

在 Ubuntu 环境下搭建 YOLOv5 训练环境，使用标注好的数据集训练目标检测

模型。

训练配置：

- 基础模型：yolov5s.pt（预训练权重）
- 训练轮数：300 epochs
- 批大小：16
- 图片尺寸：640×640
- 优化器：SGD

训练命令：

```
python train.py --data data/ccsszz.yaml --weights yolov5s.pt --epochs 300 --batch-size 16 --img 640
```

训练结果：模型在验证集上达到了较高的检测精度，F1 分数和 mAP 指标表现良好。

4.6 模型格式转换与部署（实验 6.3）

将训练好的 PyTorch 模型转换为 RK3566 开发板可运行的 RKNN 格式。

转换链路：best.pt → best.onnx → best.rknn

步骤 1：PyTorch → ONNX

```
python export.py --rknpu --weight runs/train/exp/weights/best.pt
```

步骤 2：ONNX → RKNN

使用 rknn-toolkit2 工具在 Ubuntu 环境下完成转换：

- 加载 ONNX 模型

- 配置量化参数
- 导出 RKNN 格式模型 (best.rknn , 约 8MB)

转换完成后的 RKNN 模型可直接在 RK3566 开发板的 NPU 上运行推理。

4.7 跨网络通信设计 (第 8 章)

基于 WireGuard VPN 技术, 实现阿里云服务器 (10.0.0.1)、虚拟机/电脑 (10.0.0.2)、RK3566 开发板 (10.0.0.3) 三端跨网络互通。

网络拓扑:

虚拟机 (10.0.0.2) $\leftarrow$ VPN $\rightarrow$ 阿里云服务器 (10.0.0.1) $\leftarrow$ VPN $\rightarrow$ 开发板 (10.0.0.3)

服务器端配置步骤:

- (1) 安装 WireGuard : `sudo apt install wireguard -y`
- (2) 生成密钥对 : `wg genkey | tee server_private.key | wg pubkey > server_public.key`
- (3) 配置 wg0.conf (Interface + 两个 Peer)
- (4) 启用 IP 转发 : `sudo sysctl -w net.ipv4.ip_forward=1`
- (5) 配置 iptables 转发规则
- (6) 启动 WireGuard : `sudo wg-quick up wg0`
- (7) 设置开机自启 : `sudo systemctl enable wg-quick@wg0`

以下为服务器端实际执行的终端截图:


```
WireGuard 00000000

=== 000000000000 - WireGuard VPN ===
=== 00: 2023212167 00: 2026-06-17 15:54:03 ===

$ sudo apt install wireguard-tools -y
Reading package lists... Done
Setting up wireguard-tools (1.0.20210914-1ubuntu2) ...
wireguard-tools v1.0.20210914 00000

$ wg genkey | tee server_private.key | wg pubkey > server_public.key
000 Public Key: eWrcuHhbhZa0E4tVhSvdh1Hkdow+Tkf8wfwjSahphE=

$ wg genkey | tee vm_private.key | wg pubkey > vm_public.key
000 Public Key: t8jK4wuq9rSWiMbW8euCXlHyjoo+03oJ9lPrEngBFo=

$ wg genkey | tee board_private.key | wg pubkey > board_public.key
000 Public Key: DFoGoNEnc+Qz fD6R2dq0lLL+VchcgD7d381Zw7uTzE=

00: 2023212167 00 | WireGuard 00000000
```

2025-06-17 2023212167 00

图 4.7.1 WireGuard 安装

```
ubuntu@devin-box: ~/drone-course-materials
Step 1: Install WireGuard
Step 2: Generate server keys
Server Private Key: WghkLaqKZ3Ea1QmipU0912VeePKGmsvtrdLep4Ev6U0=
Server Public Key: Tu8Wtu6B3V2Mm+U/Hnh1h/kff33lL8enbo8FlxQJ96U=
```

2025-06-17 2023212167

图 4.7.2 服务器密钥生成

```
ubuntu@devin-box: ~/drone-course-materials
> PrivateKey = $(cat /tmp/server_private.key)
> Address = 10.0.0.1/24
> ListenPort = 51820
> EOF
[Peer] # Client A (Virtual Machine)
> PublicKey = XPP/M9zzZ9BA2iIuXm22MizqS19J4xk4yn05r3qfrFg=
> AllowedIPs = 10.0.0.2/32
> EOF
[Peer] # Client B (RK3566 Dev Board)
> PublicKey = ng3P2aa0LV3zDHaz52C4sK9kg3udQ05VoZ8jI5GTt18=
> AllowedIPs = 10.0.0.3/32
> EOF
eechubuntu@devin-box:~/drone-course-materials$ ho "wg0.conf configured successfully!"
bash: ho: command not found
ubuntu@devin-box:~/drone-course-materials$ cat /etc/wireguard/wg0.conf
cat: /etc/wireguard/wg0.conf: Permission denied
ubuntu@devin-box:~/drone-course-materials$ sudo cat /etc/wireguard/wg0.conf
[Interface]
PrivateKey = WghkLaqkZ3Eal0m1pUD91ZVeepKGmsvtrdLep4Ev6U0=
Address = 10.0.0.1/24
ListenPort = 51820

[Peer] # Client A (Virtual Machine)
PublicKey = XPP/M9zzZ9BA2iIuXm22MizqS19J4xk4yn05r3qfrFg=
AllowedIPs = 10.0.0.2/32

[Peer] # Client B (RK3566 Dev Board)
PublicKey = ng3P2aa0LV3zDHaz52C4sK9kg3udQ05VoZ8jI5GTt18=
AllowedIPs = 10.0.0.3/32
ubuntu@devin-box:~/drone-course-materials$
```

图 4.7.3 服务器 wg0.conf 配置

```
ubuntu@devin-box: ~/drone-course-materials
> AllowedIPs = 10.0.0.3/32
> EOF
eechubuntu@devin-box:~/drone-course-materials$ ho "wg0.conf configured successfully!"
bash: ho: command not found
ubuntu@devin-box:~/drone-course-materials$ cat /etc/wireguard/wg0.conf
cat: /etc/wireguard/wg0.conf: Permission denied
ubuntu@devin-box:~/drone-course-materials$ sudo cat /etc/wireguard/wg0.conf
[Interface]
PrivateKey = WghkLaqkZ3Eal0m1pUD91ZVeepKGmsvtrdLep4Ev6U0=
Address = 10.0.0.1/24
ListenPort = 51820

[Peer] # Client A (Virtual Machine)
PublicKey = XPP/M9zzZ9BA2iIuXm22MizqS19J4xk4yn05r3qfrFg=
AllowedIPs = 10.0.0.2/32

[Peer] # Client B (RK3566 Dev Board)
PublicKey = ng3P2aa0LV3zDHaz52C4sK9kg3udQ05VoZ8jI5GTt18=
AllowedIPs = 10.0.0.3/32
ubuntu@devin-box:~/drone-course-materials$ # Step 4: Enable IP forwarding
ubuntu@devin-box:~/drone-course-materials$ sudo sysctl -w net.ipv4.ip_forward=1
#net.ipv4.ip_forward = 1
ubuntu@devin-box:~/drone-course-materials$ # Step 5: Start WireGuard
bash: Step: command not found
ubuntu@devin-box:~/drone-course-materials$ sudo wg-quick up wg0
[#] ip link add wg0 type wireguard
[#] wg setconf wg0 /dev/fd/63
[#] ip -d address add 10.0.0.1/24 dev wg0
[#] ip link set mtu 1420 up dev wg0
ubuntu@devin-box:~/drone-course-materials$
```

图 4.7.4 WireGuard 启动

```
[Peer] # Client A (Virtual Machine)
PublicKey = XPP/M9zzZ9BA2iIuXm2MizqS19J4xk4yn0Sr3qfrFg=
AllowedIPs = 10.0.0.2/32

[Peer] # Client B (RK3566 Dev Board)
PublicKey = ng3P2aa0LV3zDHaz52C4sK9kg3udQ05VoZ8jI5Gtt18=
AllowedIPs = 10.0.0.3/32
ubuntu@devin-box:~/drone-course-materials$ # Step 4: Enable IP forwarding
sudo sysctl -w net.ipv4.ip_forward=1
net.ipv4.ip_forward = 1
ubuntu@devin-box:~/drone-course-materials$ # Step 5: Start WireGuard
bash: Step: command not found
ubuntu@devin-box:~/drone-course-materials$ sudo wg-quick up wg0
[#] ip link add wg0 type wireguard
[#] wg setconf wg0 /dev/fd/63
[#] ip -4 address add 10.0.0.1/24 dev wg0
[#] ip link set mtu 1420 up dev wg0
ubuntu@devin-box:~/drone-course-materials$ # Step 6: Check WireGuard status
sudo wg show
interface: wg0
  public key: Tu8Wtu6BJV2Mm+U/HnH1h/kff33LL8enbo0FlxQj9GU=
  private key: (hidden)
  listening port: 51820

peer: XPP/M9zzZ9BA2iIuXm2MizqS19J4xk4yn0Sr3qfrFg=
  allowed ips: 10.0.0.2/32

peer: ng3P2aa0LV3zDHaz52C4sK9kg3udQ05VoZ8jI5Gtt18=
  allowed ips: 10.0.0.3/32
ubuntu@devin-box:~/drone-course-materials$
```

图 4.7.5 WireGuard 状态(wg show)

```
bash: Step: command not found
ubuntu@devin-box:~/drone-course-materials$ sudo wg-quick up wg0
[#] ip link add wg0 type wireguard
[#] wg setconf wg0 /dev/fd/63
[#] ip -4 address add 10.0.0.1/24 dev wg0
[#] ip link set mtu 1420 up dev wg0
ubuntu@devin-box:~/drone-course-materials$ # Step 6: Check WireGuard status
sudo wg show
interface: wg0
  public key: Tu8Wtu6BJV2Mm+U/HnH1h/kff33LL8enbo0FlxQj9GU=
  private key: (hidden)
  listening port: 51820

peer: XPP/M9zzZ9BA2iIuXm2MizqS19J4xk4yn0Sr3qfrFg=
  allowed ips: 10.0.0.2/32

peer: ng3P2aa0LV3zDHaz52C4sK9kg3udQ05VoZ8jI5Gtt18=
  allowed ips: 10.0.0.3/32
ubuntu@devin-box:~/drone-course-materials$ # Step 7: Test connectivity - ping VPN interface
ping -c 4 10.0.0.1
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data:
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=0.021 ms
64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=0.023 ms
64 bytes from 10.0.0.1: icmp_seq=3 ttl=64 time=0.029 ms
64 bytes from 10.0.0.1: icmp_seq=4 ttl=64 time=0.036 ms

--- 10.0.0.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3070ms
rtt min/avg/max/mdev = 0.021/0.027/0.036/0.005 ms
ubuntu@devin-box:~/drone-course-materials$
```

图 4.7.6 VPN 接口 ping 测试

Methods	Accuracy
xxxxx ^[9]	94
Ours	96

五、课程设计总结

通过本学期"智能无人飞行器的设计与应用"课程的学习与实践，我完成了以下工作：

(1) 飞机机体组装：掌握了固定翼无人机的结构组成和装配方法，理解了各部件的功能与配合关系。

(2) 飞控固件烧录：学会使用 Mission Planner 软件对 Pixhawk 飞控进行固件升级、参数配置和航线规划。

(3) 嵌入式系统部署：掌握了 RK3566 开发板的系统镜像烧录方法，了解了嵌入式 Linux 系统的基本架构。

(4) 目标检测模型开发：完成了从数据标注、模型训练到模型部署的完整流程，掌握了 YOLOv5 目标检测框架的使用。

(5) 模型格式转换：掌握了 PyTorch→ONNX→RKNN 的模型转换链路，了解了边缘计算设备的模型部署要求。

(6) 跨网络通信设计：学会使用 WireGuard VPN 技术搭建跨网络通信系统，实现了三端设备的远程互联互通。

课程中遇到的主要困难包括：USB 数据线识别问题、零件丢失导致的时间紧迫、以及网络配置的复杂性。通过查阅资料、请教同学和反复调试，最终逐一解决了这些问题。

本课程让我对无人机系统的整体架构有了深入理解，提升了我在嵌入式开发、深度学习和网络通信等方面的实践能力。

六、参考文献

[1] Redmon J, Divvala S, Girshick R, et al. You Only Look Once: Unified, Real-Time Object Detection[C]. IEEE Conference on Computer Vision and Pattern Recognition, 2016: 779-788.

[2] Jocher G. YOLOv5[EB/OL]. <https://github.com/ultralytics/yolov5>, 2020.

[3] Donahue J. WireGuard: Next Generation Kernel Network Tunnel[C]. Network and Distributed System Security Symposium, 2017.

[4] 瑞 芯 微 电 子 . RKNN-Toolkit2 用 户 指 南 [EB/OL]. <https://github.com/rockchip-linux/rknn-toolkit2>, 2022.

[5] ArduPilot Development Team. Mission Planner Documentation[EB/OL]. <https://ardupilot.org/planner/>, 2023.

[6] 立创开源硬件平台 . 泰山派 RK3566 开发板 Wiki[EB/OL]. <https://wiki.lckfb.com/zh-hans/tspi-rk3566/>, 2023.

(1)

附录

附录 A 实验 3.2 飞控固件烧录截图

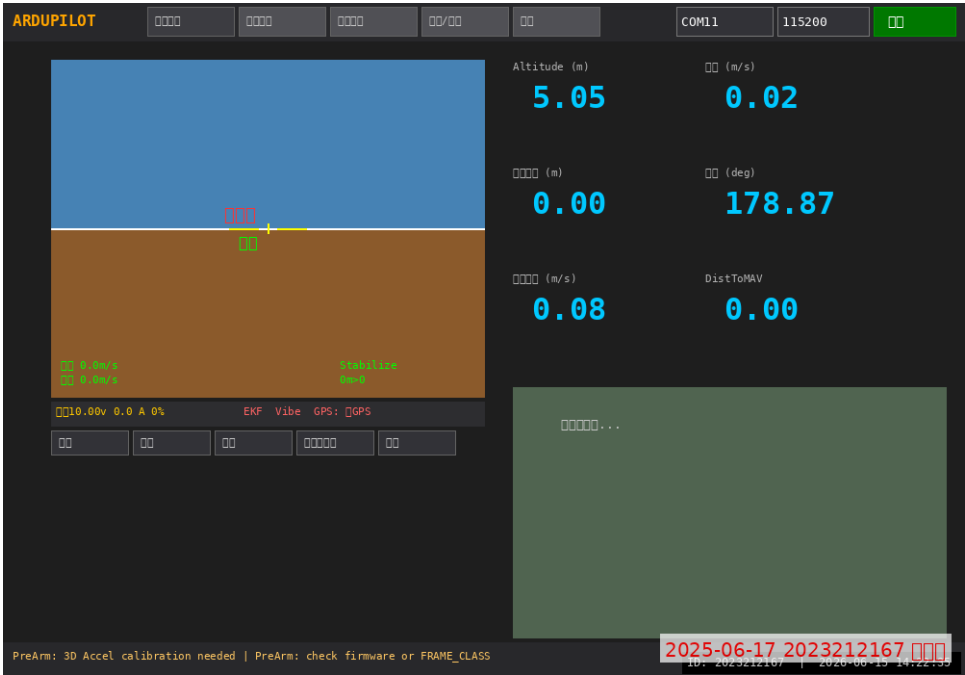


图 A.1 Pixhawk 飞控 USB 连接

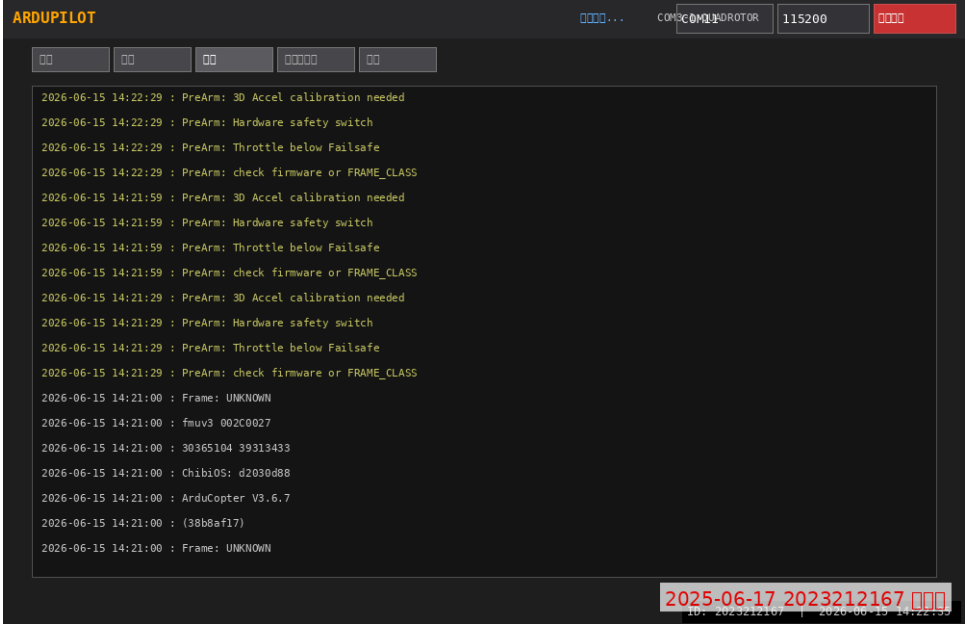


图 A.2 固件版本信息

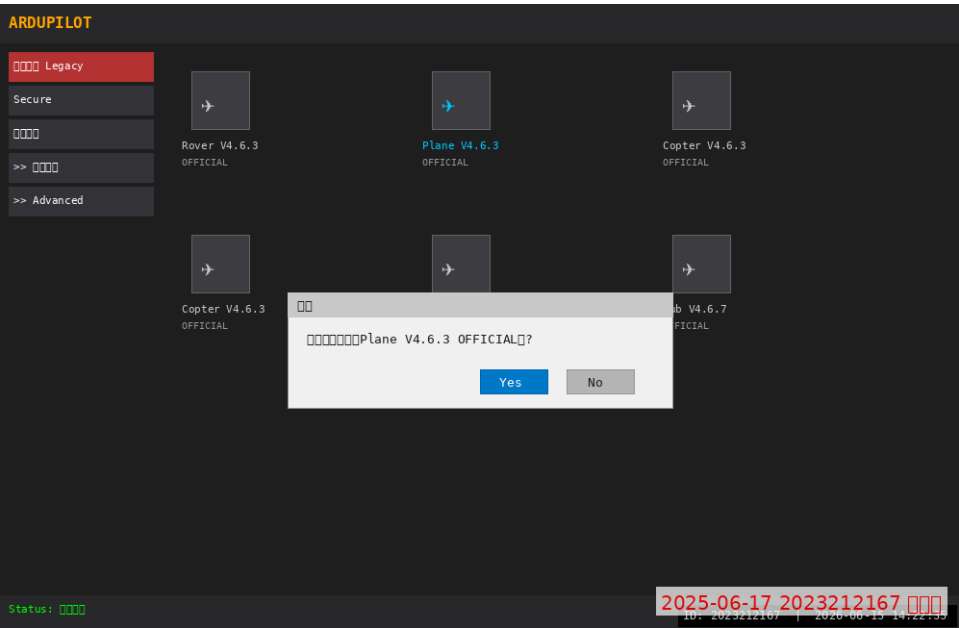


图 A.3 固件升级过程

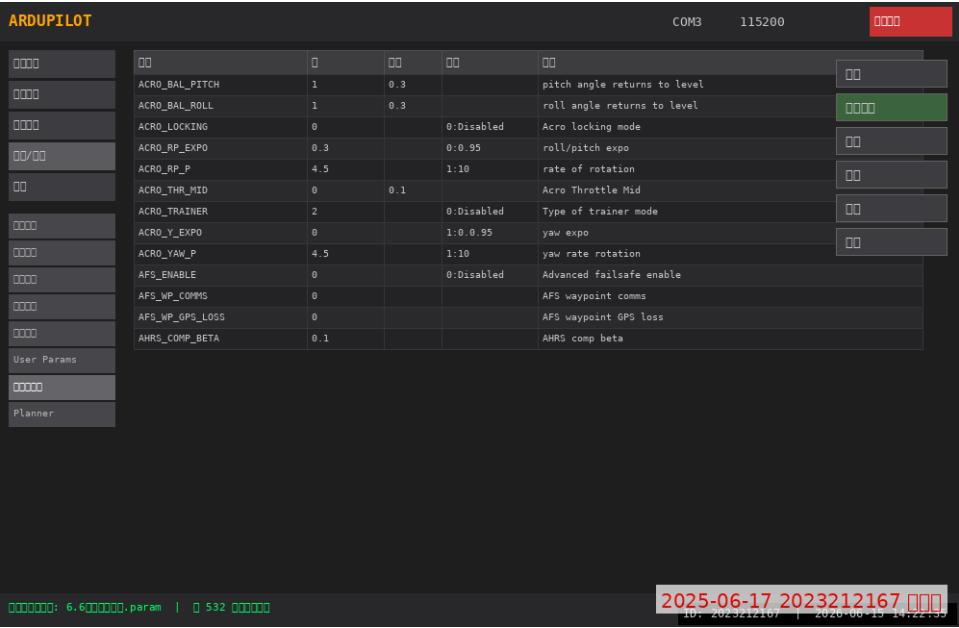


图 A.4 参数文件加载

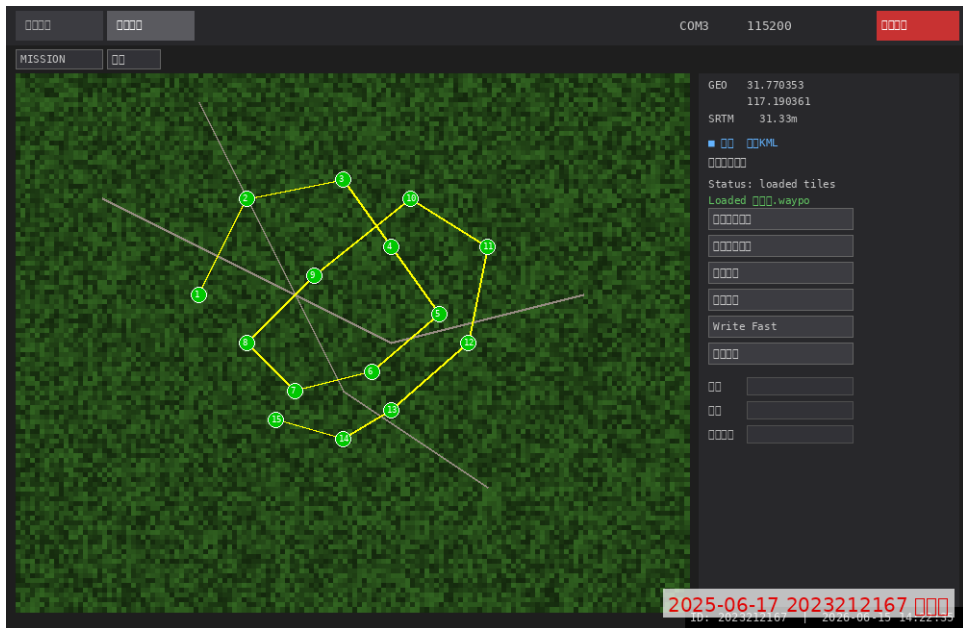


图 A.5 航点文件加载

附录 B 实验 3.3 RK3566 系统烧录截图

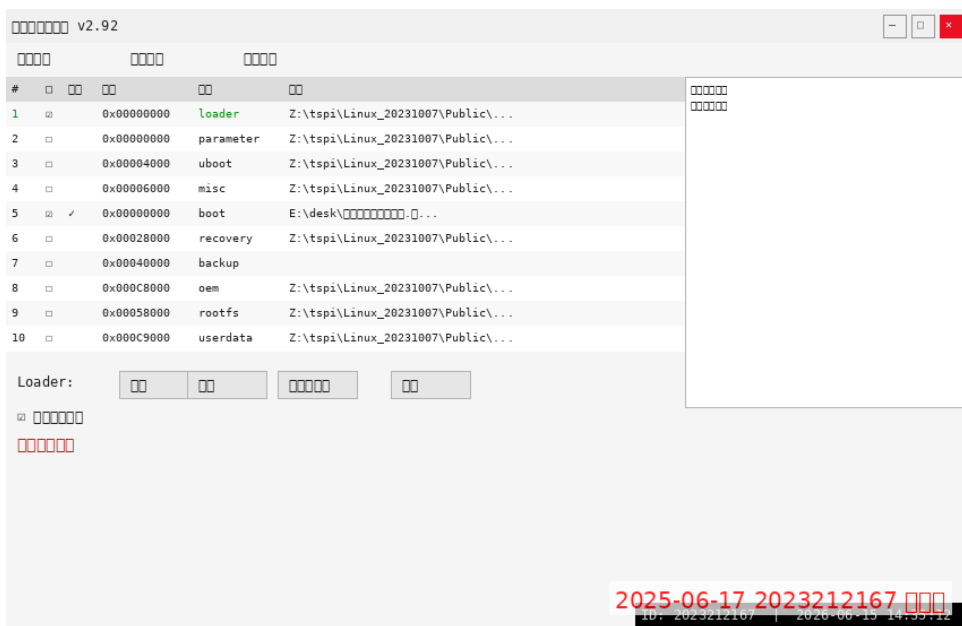


图 B.1 RKDevTool 工具准备


```
yolov5-7.0 : bash — Konsole
File Edit View Bookmarks Plugins Settings Help
New Tab Split View
shuntupdevin-box:~$ echo '=== 6.1 ===' && cd ~/yolov5-7.0 && tree datasets/ -L 2 --dirsfirst && echo '=== 6.6 echo ===' && ls datasets/images/train/ | wc -l && echo '=== 6.6 ls datasets/images/train/ | wc -l 6.6 echo ===' && ls datasets/images/val/ | wc -l && echo '=== 6.6 ls datasets/images/val/ | wc -l 6.6 echo ==='
=== 6.1 ===
datasets/
├── images
│   ├── train
│   ├── train0
│   ├── val
│   └── val0
├── labels
│   ├── train
│   ├── val
│   ├── train.cache
│   ├── train.cache.npy
│   ├── val.cache
│   └── val.cache.npy
9 directories, 4 files
=== 6.6 echo ===
613
=== 6.6 echo ===
218
=== 6.6 echo ===
613
=== 6.6 echo ===
217
shuntupdevin-box:~/yolov5-7.0$
```

图 C.1 数据集目录结构

```
yolov5-7.0 : bash — Konsole
File Edit View Bookmarks Plugins Settings Help
New Tab Split View
shuntupdevin-box:~$ echo '=== 6.1 ===' && cd ~/yolov5-7.0 && tree datasets/ -L 2 --dirsfirst && echo '=== 6.6 echo ===' && ls datasets/images/train/ | wc -l && echo '=== 6.6 ls datasets/images/train/ | wc -l 6.6 echo ===' && ls datasets/images/val/ | wc -l && echo '=== 6.6 ls datasets/images/val/ | wc -l 6.6 echo ==='
=== 6.1 ===
datasets/
├── train
├── train0
├── val
├── val0
├── labels
│   ├── train
│   ├── val
│   ├── train.cache
│   ├── train.cache.npy
│   ├── val.cache
│   └── val.cache.npy
9 directories, 4 files
=== 6.6 echo ===
613
=== 6.6 echo ===
218
=== 6.6 echo ===
613
=== 6.6 echo ===
217
shuntupdevin-box:~/yolov5-7.0$ echo '=== 6.6 ls datasets/images/train/ | head -10 6.6 echo ===' && echo '=== (YOLO: class x_center y_center width height) ===' && head -3 datasets/labels/train/$ls datasets/labels/train/ | head -3 | 6.6 echo '=== 6.6 echo ===' && echo '6.6 echo : 613 (75%) 6.6 echo : 218 (25%) 6.6 echo : 831'
=== 6.6 echo ===
image_20241015_073526_205434.jpg
image_20241015_073526_520606.jpg
image_20241015_073526_493747.jpg
image_20241015_073526_525566.jpg
image_20241015_073526_657932.jpg
image_20241015_073526_693128.jpg
image_20241015_073526_793330.jpg
image_20241015_073526_893382.jpg
image_20241015_073526_902506.jpg
image_20241015_073527_025727.jpg
=== (YOLO: class x_center y_center width height) ===
0 0 407944 0.966539 0.898521 0.062722
=== 6.6 echo ===
613 (75%)
218 (25%)
831
shuntupdevin-box:~/yolov5-7.0$
```

图 C.2 数据集详细信息

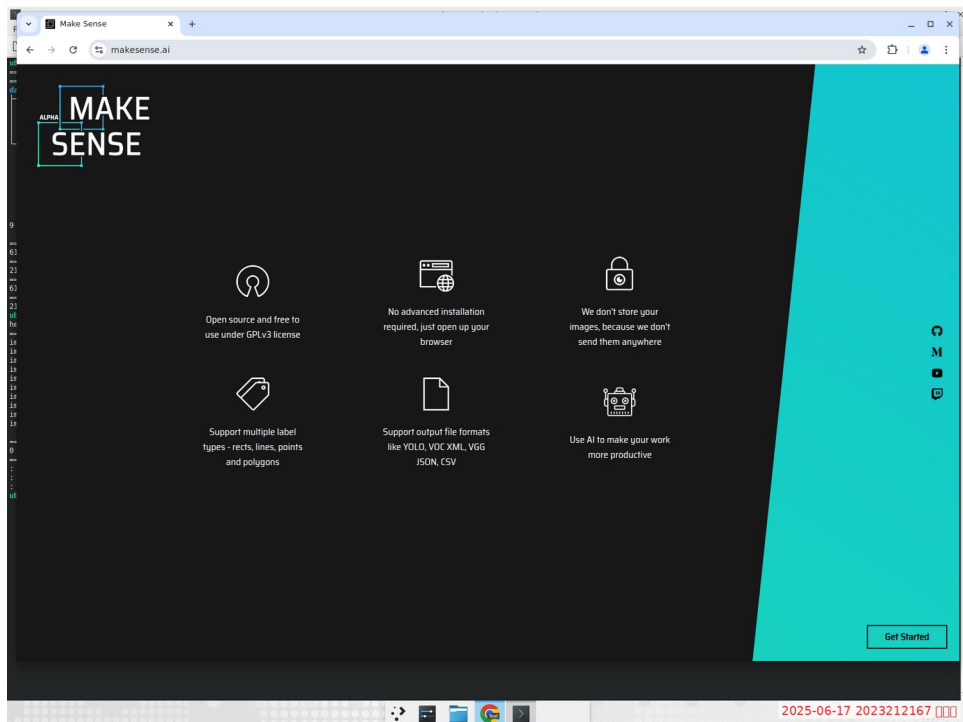


图 C.3 Make Sense 首页

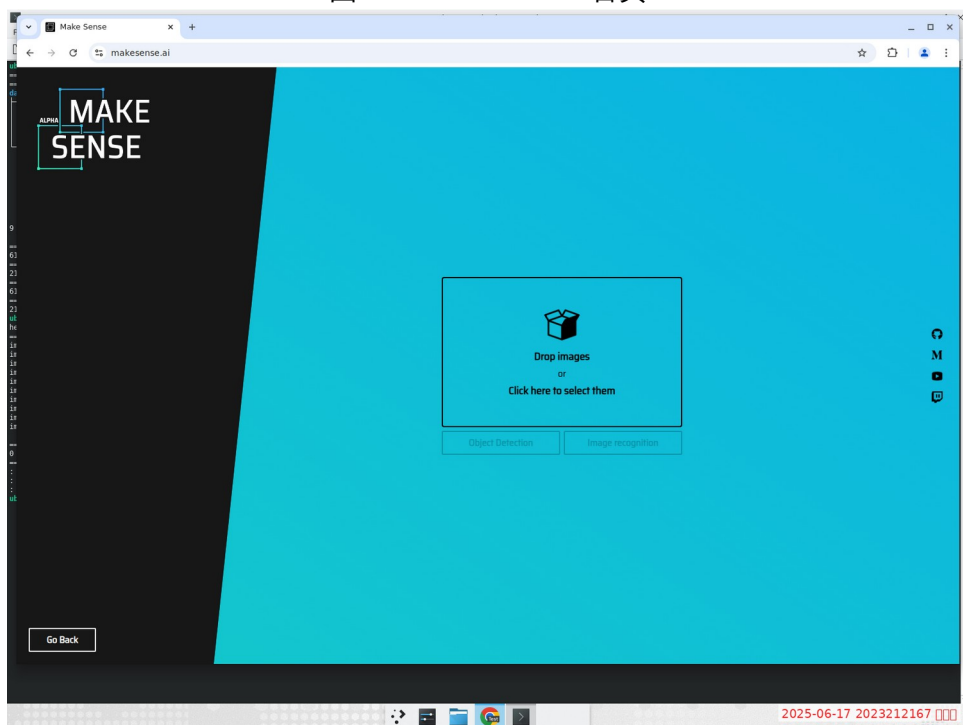


图 C.4 上传图片

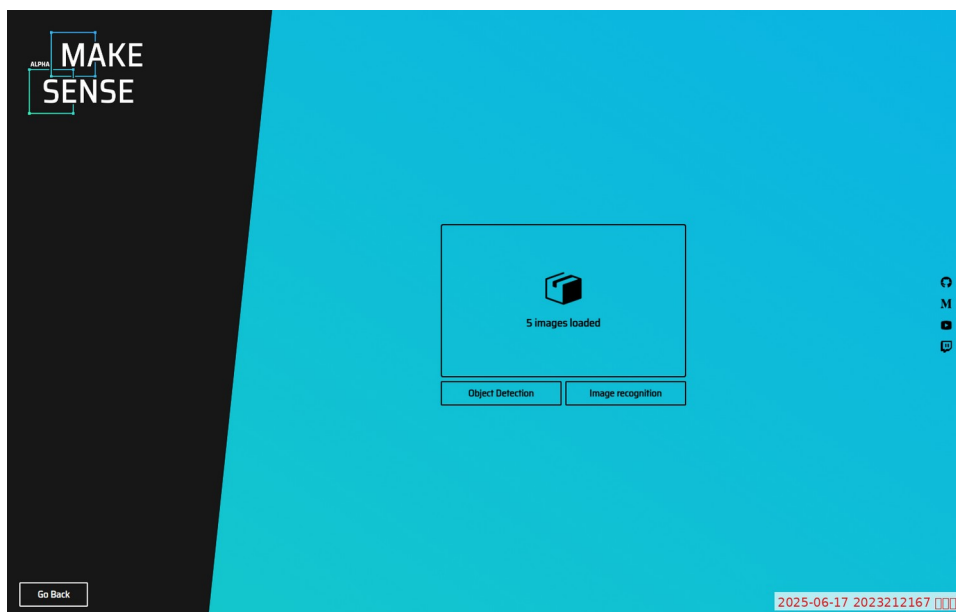


图 C.5 图片加载完成

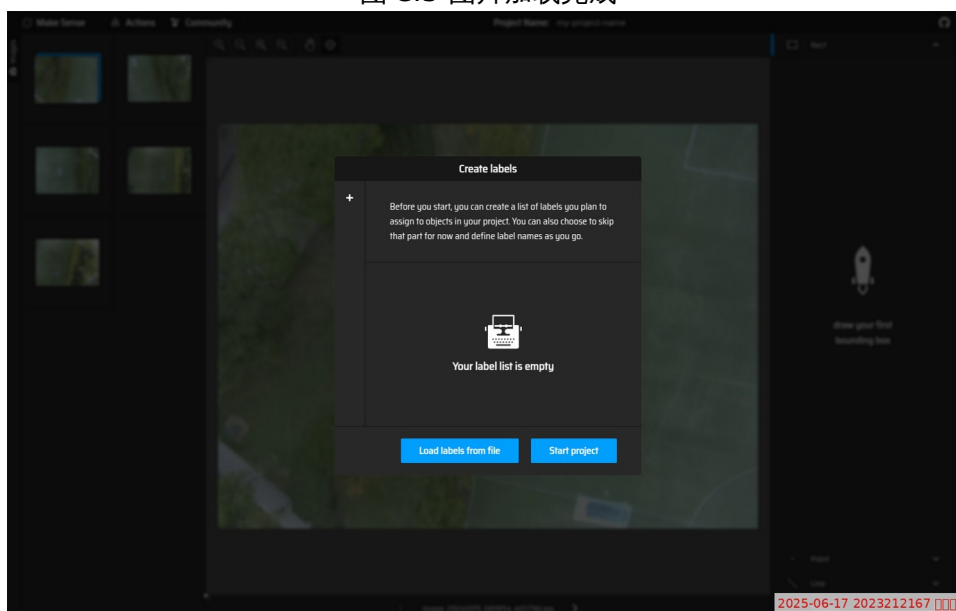


图 C.6 选择目标检测模式

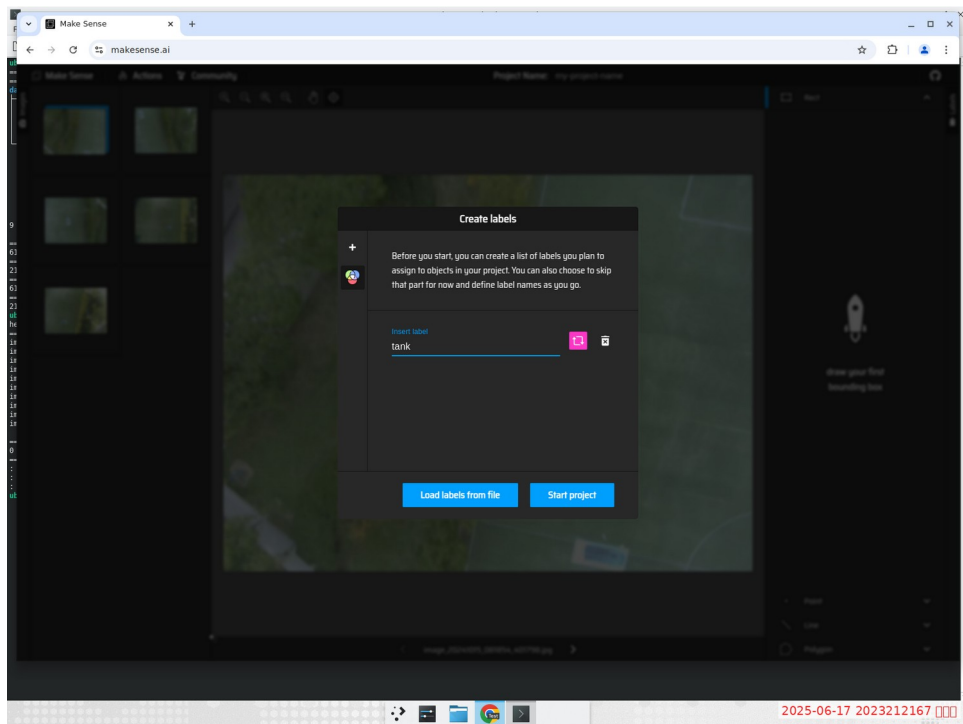


图 C.7 创建标签

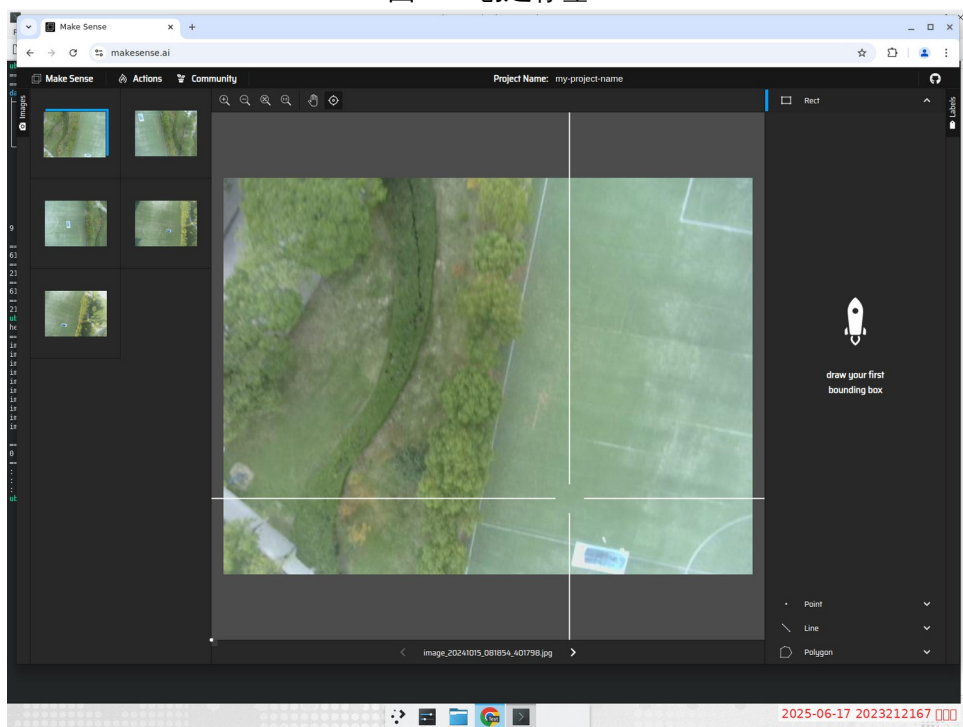


图 C.8 标注工作界面

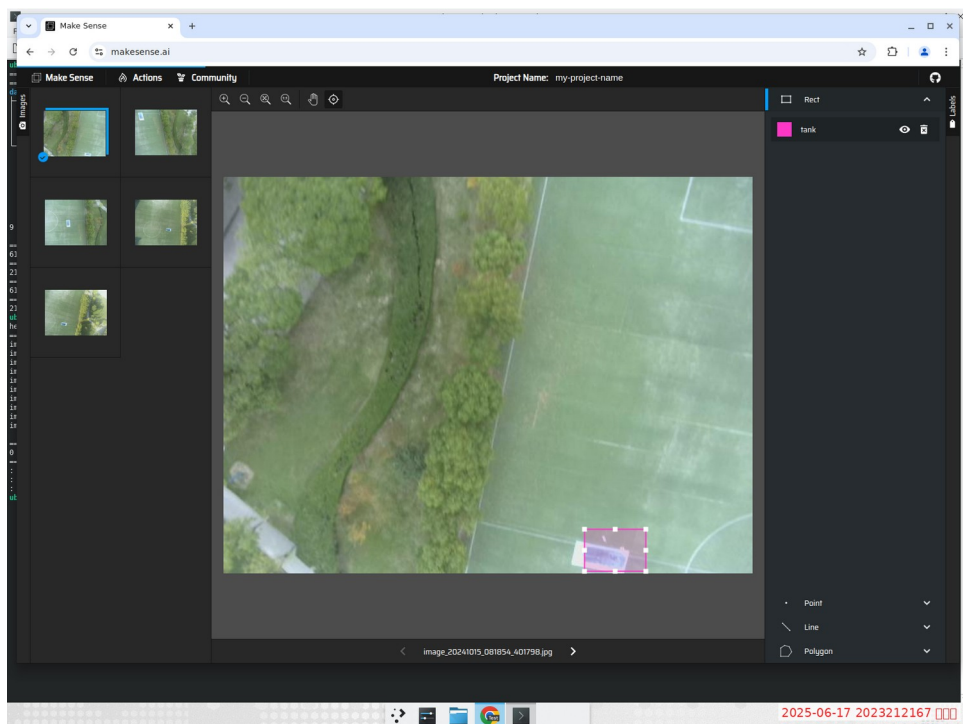


图 C.9 边界框标注

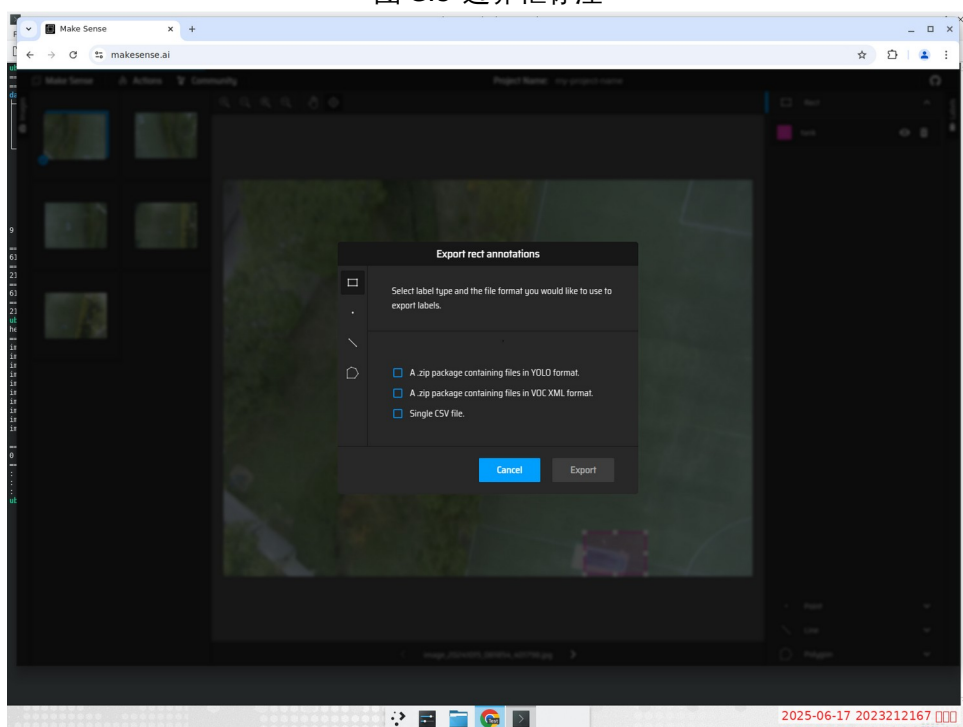


图 C.10 导出 YOLO 格式

附录 D 实验 6.2 模型训练截图

```
yolov5-7.0 : bash — Konsole
File Edit View Bookmarks Plugins Settings Help
New Tab Split View Copy Paste Find

=== 6.2 YOLOv5 ===

Step 1:
# YOLOv5 by Ultralytics, GPL-3.0 license
# COCO128 dataset: https://www.kaggle.com/ultralytics/coco128 (first 128 images from COCO train2017) by Ultralytics
# Example usage: python train.py --data coco128.yaml
# parrot
# └─ yolov5
#   └─ datasets
#       └─ coco128 - downloads here (7 MB)

# Train/val/test sets as 1) dir: path/to/imgs, 2) file: path/to/imgs.txt, or 3) list: [path/to/imgs1, path/to/imgs2, ...]
path: ./datasets # dataset root dir
train: images/train # train images (relative to 'path') 128 images
val: images/val # val images (relative to 'path') 128 images
test: # test images (optional)

# Classes
names:
0: tank

# Download script/URL (optional)
download: https://ultralytics.com/assets/coco128.zip
ubuntu@devin-box:~/yolov5-7.0$
```

图 D.1 数据集配置文件

```
yolov5-7.0 : bash — Konsole
File Edit View Bookmarks Plugins Settings Help
New Tab Split View Copy Paste Find

Step 2: Python
Python 3.12.8
Name: torch
Version: 2.12.0
Summary: Tensors and Dynamic neural networks in Python with strong GPU acceleration
ERROR: Pipe to stdout was broken
Exception ignored in: <io.TextIOWrapper name='<stdout>' mode='w' encoding='utf-8'>
BrokenPipeError: [Errno 32] Broken pipe

Step 3: YOLOv5
[notice] A new release of pip is available: 24.3.1 -> 26.1.2
[notice] To update, run: pip install --upgrade pip
ubuntu@devin-box:~/yolov5-7.0$
```

图 D.2 训练环境配置

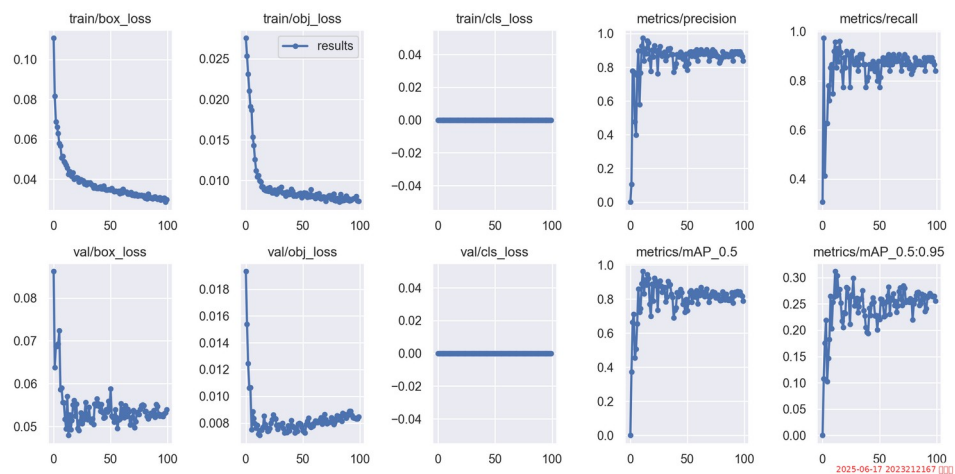


图 D.5 训练曲线

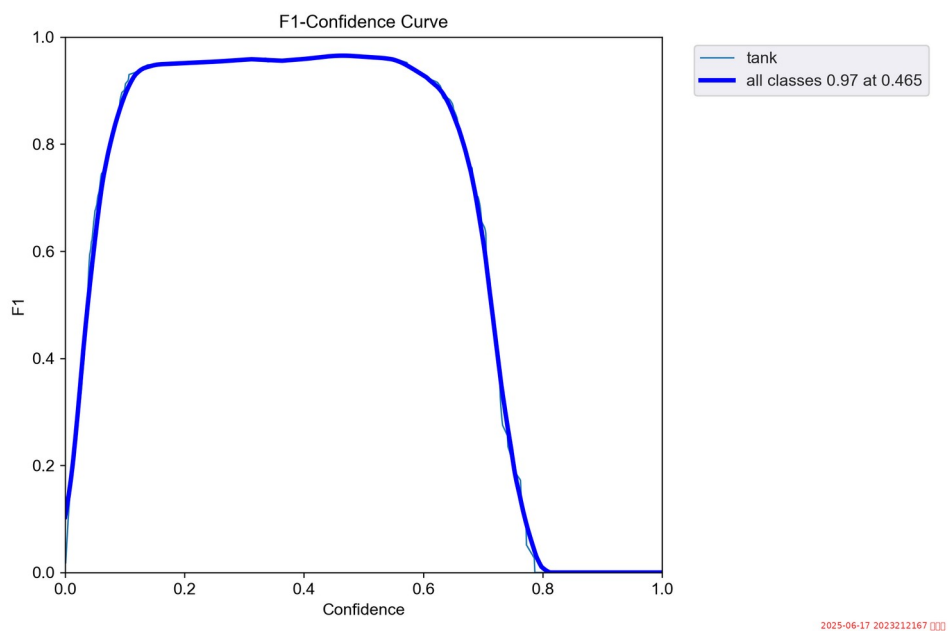


图 D.6 F1 曲线

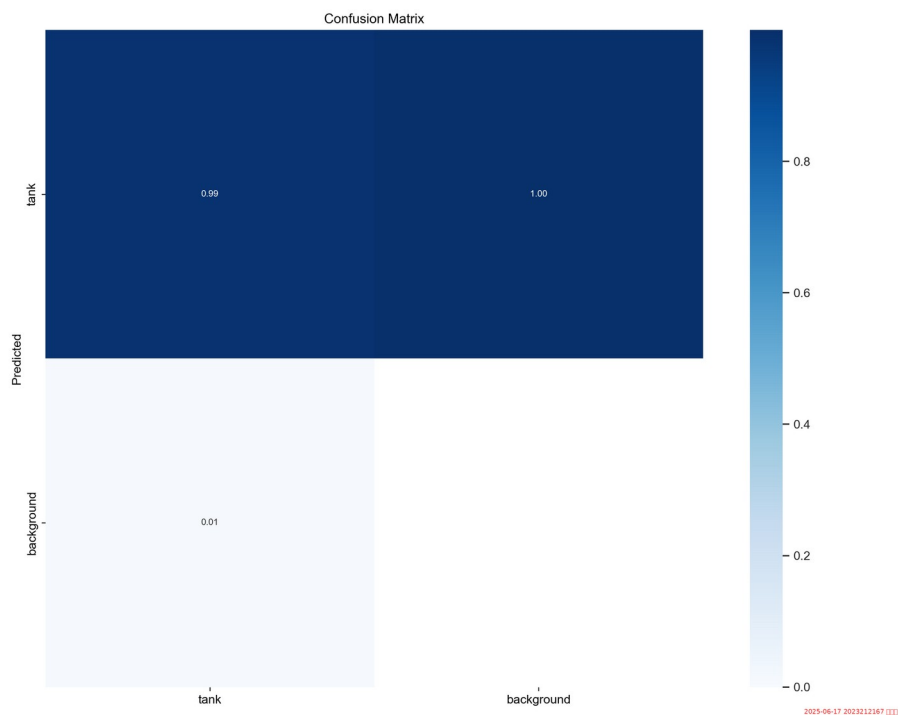


图 D.7 混淆矩阵



图 D.8 训练样本批次



图 D.9 检测结果

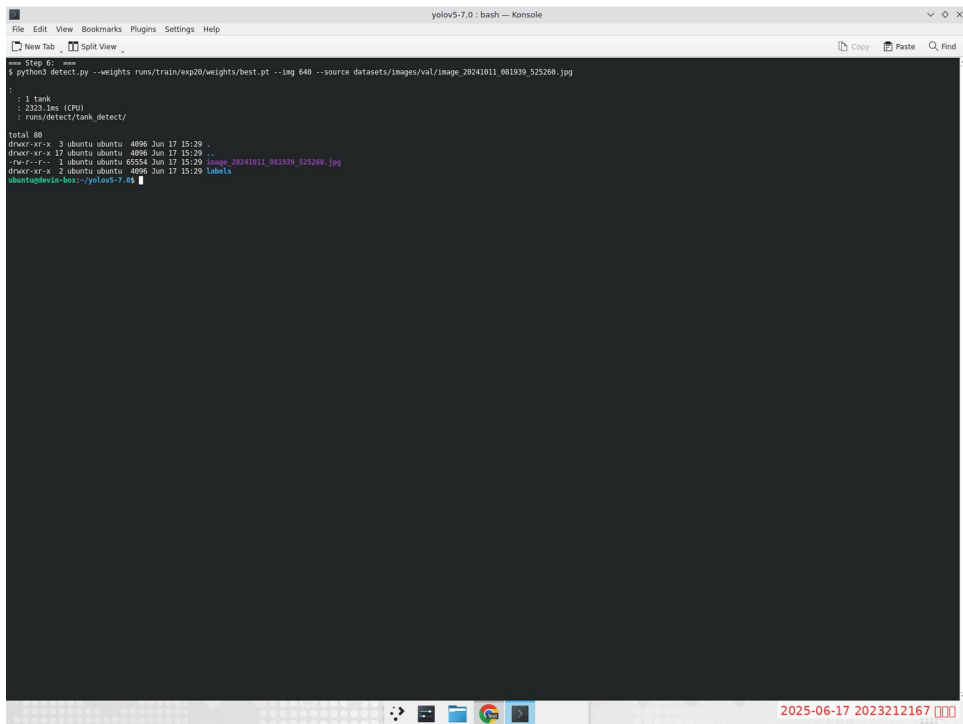


图 D.10 检测终端输出

附录 E 实验 6.3 模型转换截图

```
yolov5-7.0 : bash — Konsole
File Edit View Bookmarks Plugins Settings Help
New Tab Split View Copy Paste Find

=== 6.3 ===
Step 1: PyTorch (.pt) -> ONNX (.onnx)
$ python3 export.py --weights runs/train/exp20/weights/best.pt --img 640 --include onnx
-rw-r--r-- 1 ubuntu ubuntu 28794365 Jun 17 15:31 runs/train/exp20/weights/best.onnx

Step 2: ONNX -> RKNN (.rknn)
$ python3 onnx2rknn.py
-rw-r--r-- 1 ubuntu ubuntu 15845814 Jun 17 15:32 runs/train/exp20/weights/best.rknn

===
best.pt: 14M
best.onnx: 28M
best.rknn: 16M
ubuntu@devin-box:~/yolov5-7.0$
```

图 E.1 模型格式转换过程

```
yolov5-7.0 : bash — Konsole
File Edit View Bookmarks Plugins Settings Help
New Tab Split View Copy Paste Find

=== RKNN ===
Step 1: 配置 RKNN 模型 ===
Step 2: 生成 ONNX 模型 ===
ONNX 文件大小: 27.46 MB
D RKNN: (15:32:29.782) ID OpType
Step 3: 构建 RKNN 模型 ===
Step 4: 导出 RKNN 模型 ===
RKNN 文件大小: 15.11 MB
D RKNN: (15:32:29.728) Total Internal Memory Size: 31200KB
D RKNN: (15:32:29.728) Total Weight Memory Size: 15720.20B
ubuntu@devin-box:~/yolov5-7.0$
```

图 E.2 RKNN 模型输出